



Démarrer & présenter Motus

Docker expliqué simplement · les commandes de la démo · Minikube — M2 MIAGE SITN

1 Docker, c'est quoi ? (en simple)

L'idée

Docker met une application **et tout ce dont elle a besoin** (Java, dépendances, config) dans une **boîte** autonome. Cette boîte tourne **pareil partout** — sur ton Mac, celui de Steven, ou un serveur. Fini le « ça marche chez moi mais pas chez toi ».

Les 4 mots à connaître

- **Image** : le « moule » figé de l'appli (recette + contenu).
- **Conteneur** : une image **qui tourne** (une boîte allumée).
- **Dockerfile** : la **recette** qui dit comment fabriquer l'image.
- **Volume** : un **disque** à part pour garder les données (ex. MySQL) même si la boîte est supprimée.

docker compose

Un seul fichier `docker-compose.yml` décrit **plusieurs conteneurs** et comment ils se parlent. Une commande démarre **tout** : MySQL + les 4 services. Ils se trouvent par leur **nom** (ex. `mysql`, `dict-service`) sur un réseau privé.

Comment NOTRE projet l'utilise

Chaque service a un **Dockerfile** « multi-étapes » : il **compile** le code avec Maven (JDK 26) puis met juste le `.jar` dans une petite image (JRE 26). `docker-compose.yml` lance **1 MySQL** (une base par service) + **player, game, dict, score**.

Pourquoi c'est pratique pour la soutenance : tu démarres **toute** l'appli (4 services + base) en **une commande**, sans installer Java ni MySQL sur la machine de démo. Tout est dans les conteneurs.

2 Démarrer la démo (le jour J)

Le plus simple — double-clic

- 1 Ouvre **Docker Desktop** (l'icône baleine 🐳 doit être « running »).
- 2 Double-clique `lancer-demo.command` → il démarre Docker si besoin, construit et lance le back, et sert le front.
- 3 Attends ~30 s, puis ouvre `http://localhost:5500` dans le navigateur. Joue !
- 4 Pour tout arrêter : double-clique `arreter-demo.command`.

La version « à la main » (si on te pose la question)

```
# 1) démarrer tout le back (MySQL + les 4 services). --build la 1re fois seulement
docker compose up --build -d

# 2) servir le front (page web) sur le port 5500
cd frontend && python3 -m http.server 5500

# 3) ouvrir dans le navigateur
http://localhost:5500

# 4) tout arrêter (les scores sont gardés)
docker compose down
# ... ou tout effacer pour repartir de zéro (supprime les données) :
docker compose down -v
```

1er démarrage plus lent : Docker télécharge/compile les images (quelques minutes). Les fois suivantes c'est quasi instantané. **dict-service** charge 132 000 mots au démarrage → laisse-lui ~30 s avant de jouer.

3 Commandes Docker utiles

Commande	Ce qu'elle fait
<code>docker compose ps</code>	Voir les conteneurs du projet et leur état
<code>docker compose logs -f dict-service</code>	Suivre les logs d'un service en direct
<code>docker ps</code>	Tous les conteneurs qui tournent
<code>docker images</code>	Les images construites
<code>docker compose up --build -d</code>	(Re)construire et démarrer en arrière-plan
<code>docker compose restart game-service</code>	Redémarrer un seul service
<code>docker compose down -v</code>	Tout arrêter et effacer les données (volume)

Dépannage express : « Cannot connect to the Docker daemon » → Docker Desktop n'est pas lancé. « port is already allocated » → un vieux conteneur tourne, fais `docker compose down`. Un service redémarre en boucle → `docker compose logs <service>` pour voir l'erreur (souvent MySQL pas encore prêt : ça se stabilise tout seul).

4 Version Kubernetes / Minikube

La même appli, en « cloud-native »

docker-compose = orchestrer les conteneurs **en local**. **Minikube** = les orchestrer avec **Kubernetes** (un cluster sur ta machine), qui ajoute le **scaling automatique**, l'**auto-réparation** des services qui tombent, et la **découverte** par nom. Les fichiers sont dans [k8s/](#) (voir [k8s/README.md](#)).

```
# tout déployer sur Minikube (1 commande)
bash k8s/deploy.sh
# rendre l'appli accessible au front (2e terminal), puis http://localhost:5500
bash k8s/port-forward.sh

# DÉMOS qui impressionnent le jury :
kubectl get pods                # l'état du cluster
kubectl scale deployment/dict-service --replicas=3  # scalabilité horizontale
kubectl delete pod <un-pod>      # auto-réparation : K8s le recrée seul
kubectl get hpa                  # auto-scaling selon le CPU
```